Riverbend — GCP Deployment

Infrastructure-as-code lives in `infra/terraform/` (validated: `terraform fmt -check` + `validate` in CI). It provisions a **managed, container-based** deployment — the simplest *credible* production shape for this two-process app — not a hand-run VM.

Status (be precise):

- **Terraform validated** ✅ (`fmt` + `validate`) — the production LB + custom-domain + internal-ingress shape.
- **Actually deployed + end-to-end verified** ✅ on **GCP staging (`riverbend-staging`, 2026-11-06)** using a

cheaper native-Cloud-Run shape (see `docs/GCP_STAGING_DEPLOYMENT_REPORT.md`): public browser access, customer form → real gpt-5.4-mini → `$65` work order, tenant isolation (RLS), idempotency, admin/staff + staff-scoped isolation, auth, budget hold+recovery, restart durability — all passed (email NOT VERIFIED, live LLM-outage not injected). **Reproducibility verified:** the environment was later rebuilt from scratch by running only `make gcp-deploy ENV=staging` (exit 0, no manual steps, after 3 script fixes — see report §12) and is currently **running** (≈$25–35/mo; `make gcp-destroy ENV=staging CONFIRM=yes` to remove). Project + billing preserved throughout.

- **Production (LB + domain + HA)** — Terraform-validated, **not applied**; production readiness is not claimed.

The staging deploy surfaced and fixed real Terraform bugs (worker missing `ADMIN_BOOTSTRAP_TOKEN` / `LLM_MODEL` / LiteLLM keys → would crash or run on the FakeGateway; LiteLLM needed a config image + its own database). See the report §7 and the **D27** notes in `infra/terraform/runtime.tf`.

1. Chosen architecture (what the Terraform provisions)

- **Cloud Run v2** services for **api**, **worker** (the app image, `start:api` / `start:worker`), and the

internal **litellm** proxy (a **separate** digest-pinned image built from `infra/litellm/Dockerfile`, which bakes `litellm/config.yaml`). api is public via an HTTPS load balancer; worker and litellm are **internal-only** (`INGRESS_TRAFFIC_INTERNAL_ONLY`).

- **Cloud SQL (PostgreSQL)** — `google_sql_database_instance` + database; reached over the Cloud SQL

connector, never publicly exposed.

- **External HTTPS load balancer** (global address, managed SSL cert, URL map, backend service, serverless

NEG) + a **Cloud Armor** security policy — public endpoint is **HTTPS only**.

- **Artifact Registry** for the container image.
- **Secret Manager** for all secrets (DB URLs, `ADMIN_BOOTSTRAP_TOKEN`, `EMAIL_SIGNING_KEY`,

`resend-api-key`, `resend-webhook-secret`, `litellm-master-key`, `openai-api-key`), injected per service with least-privilege accessor IAM.

- **4 service accounts** (api/worker/litellm/deployer), least-privilege IAM.
- **Workload Identity Federation** pool + provider so **GitHub Actions deploys via OIDC — no long-lived JSON

keys**.

Why: Cloud Run matches the stateless api/worker processes and scales to zero; Cloud SQL is managed Postgres (our only datastore — pg-boss rides on it, so **no Redis/Memorystore needed**); keeping worker + litellm + DB private limits the attack surface; Secret Manager + WIF keep secrets and CI auth keyless.

Simpler alternative (documented, not implemented): a single Compute Engine VM running `docker compose up` with Postgres as a co-located container + a persistent disk. Lower cost/complexity for a demo, but co-locating app + DB on one VM is **not** HA — a documented MVP trade-off, not provisioned here to avoid maintaining two architectures.

2–5. APIs, permissions, config, credentials

- **Required APIs** (enabled by `google_project_service`): `run`, `sqladmin`, `artifactregistry`,

`secretmanager`, `compute`, `iam`, `iamcredentials`, `aiplatform`. (Logging/Monitoring are on by default. `aiplatform` is listed for an optional Vertex backend; the shipped path is OpenAI-via-LiteLLM and does not call it.)

- **Permissions:** the deployer SA needs Run admin, Artifact Registry writer, Cloud SQL client, Secret

Manager admin (to create versions), and `iam.serviceAccountUser`. Runtime SAs get only `secretAccessor` on their own secrets + Cloud SQL client.

- **Config:** copy `infra/terraform/terraform.tfvars.example` → `terraform.tfvars` and set

`project_id`, `region`, `domain` (for the managed cert), `github_repo` (owner/repo for WIF), `resend_from`, `resend_reply_domain`. **Never put secret values in tfvars/variables/outputs/state.**

- **Credentials:** put real secret *values* into Secret Manager out-of-band (e.g.

`gcloud secrets versions add openai-api-key --data-file=-`), not in Terraform.

6–9. Provision, build/deploy, migrate, secrets

```
cd infra/terraform
terraform init                    # configure a remote backend first — see §Terraform state
terraform plan -out tf.plan
```

```
terraform apply tf.plan            # creates registry, SQL, SAs, secrets (empty), LB, WIF

# build + push TWO images: the app (api+worker, behaviour selected by command) and the litellm proxy
# (stock berriai has no model_list → the riverbend-extraction alias fails; infra/litellm/Dockerfile bakes it).
gcloud auth configure-docker <region>-docker.pkg.dev
docker build -t <region>-docker.pkg.dev/<project>/riverbend/app:<sha> .
docker push      <region>-docker.pkg.dev/<project>/riverbend/app:<sha>
docker build -f infra/litellm/Dockerfile -t <region>-docker.pkg.dev/<project>/riverbend/litellm:<sha> .
docker push      <region>-docker.pkg.dev/<project>/riverbend/litellm:<sha>
# then set var.image_api / image_worker (app) and var.litellm_image (the custom litellm) to the @sha256 digests

# add secret VALUES (out-of-band; never in TF)
printf '%s' "$OPENAI_API_KEY" | gcloud secrets versions add openai-api-key --data-file=-
# …repeat for admin-bootstrap-token, email-signing-key, resend-api-key, resend-webhook-secret, litellm-master-k

# deploy the image to the Run services (var.image_* / gcloud run deploy)
```

- **Migrations** run automatically on **api** startup (migrate-before-serve, owner role). For many instances,

gate them behind a one-shot migration job rather than every boot (noted in not-built).

- **First admin:** `POST https://<domain>/admin/init` with the bootstrap token (single-use, D20.2).
- **DB connection strings (secret values):** api/worker reach Cloud SQL over the connector volume mounted at

`/cloudsql`, so the `database-*` secrets use a **unix-socket host**, not an IP: `postgresql://<user>:<pw>@/riverbend?host=/cloudsql/<project>:<region>:riverbend-pg`. Use the `app_user` (NOBYPASSRLS) role for `DATABASE_URL`, and the owner/migrator role only for `DATABASE_MIGRATOR_URL`.

Staging deployment (conservative, disposable)

Use a DEDICATED, EMPTY project — never an unrelated or production project. Create one so nothing existing can be touched: `gcloud projects create riverbend-staging --name="Riverbend Staging"` and link billing. Confirm the active project before every command: `gcloud config get-value project`.

`infra/terraform/terraform.staging.tfvars.example` sets small, zonal, scale-to-zero, non-protected resources (≈ half the SQL cost, near-zero idle Cloud Run). Deploy:

```
cp infra/terraform/terraform.staging.tfvars.example infra/terraform/terraform.staging.tfvars
# edit project_id/domain/image digests, build + push BOTH images first (see §6)
cd infra/terraform
terraform init
terraform plan  -var-file=terraform.staging.tfvars -out tf.plan     # read-only; review "to add/change/destroy"
terraform apply tf.plan                                             # ONLY after explicit approval
```

Cleanup / teardown (staging)

```
cd infra/terraform
terraform destroy -var-file=terraform.staging.tfvars   # db_deletion_protection=false makes the SQL instance de
# Secret Manager secrets are not auto-deleted; remove if desired:
for s in admin-bootstrap-token database-url database-admin-url database-migrator-url web-form-signing-key \
         email-signing-key resend-webhook-secret resend-api-key litellm-master-key openai-api-key; do
  gcloud secrets delete "$s" --quiet; done
# Optionally delete the whole project (fastest, removes everything incl. billing exposure):
gcloud projects delete riverbend-staging
```

10. Health & smoke tests

`GET https://<domain>/readyz` → `{"status":"ready"}`; `GET /healthz` → 200; `GET /admin/login` → 200. litellm `/health/liveliness` is internal-only. The **worker** is a background queue consumer but Cloud Run runs it as a *service*, so it binds a minimal health listener on `$PORT` (D25): the Terraform worker service has an HTTP startup + liveness probe on `/healthz` (internal-only). Then run the browser/API smoke in `docs/FINAL_MANUAL_TEST.md`.

11. Logging & monitoring

Cloud Run → Cloud Logging automatically (logs are **ids-only**, no PII/secrets — VII). Add log-based metrics

- alerts for 5xx rate, p95 latency, pg-boss backlog, and LiteLLM spend (not-built: full OTel/dashboards).

12. Backup & recovery

Enable Cloud SQL **automated backups + point-in-time recovery** and a retention window; restore via `gcloud sql backups restore`. Secret Manager versions are retained. The container image is immutable in Artifact Registry (pin by digest). No other durable state (pg-boss + all data live in Postgres).

13. Rollback

Deploy the previous image digest to the Run services (`gcloud run services update … --image <prev-digest>`) — Cloud Run keeps revisions, so traffic can be shifted back instantly. Roll DB forward-only (migrations have `down` for dev, but prefer a new corrective migration in prod).

14. Teardown

`terraform destroy` (after emptying/confirming you want to delete Cloud SQL — it holds data). Delete Secret Manager secrets separately if desired. **Terraform state:** store it in a **GCS backend with**

versioning + object lock, never in git — state can contain resource metadata (and would hold secrets if any were put in TF, which we forbid). Restrict the bucket to the deployer SA.

Optional future (managed, lower-ops) — already largely the above

This IaC *is* the managed path (Cloud Run + Cloud SQL). A further step would be managed Redis only if a non-Postgres queue were introduced (not planned), and Cloud Run jobs for scheduled sweeps. Do not add GKE/ Kubernetes — it is unnecessary for this workload.

CI/CD

`.github/workflows/ci.yml` runs on push/PR: typecheck, dependency-cruiser (module boundaries), tests (real Postgres via Testcontainers), **gitleaks** secret scan, and `terraform fmt/init/validate`. A deploy workflow should be **manually triggered**, use a **GitHub Environment** for approval, and authenticate via the **WIF** provider above (no JSON keys). Do not auto-deploy on push.